

FrazIndia — Auth Service Metadata APIs

(enterprise-level technical documentation — developer onboarding)

Project root: `frazindia-auth-service-metadata-apis/`
Main Java package: `com.fipl.reporting.authentication`
Build: Maven (`pom.xml`)
Java runtime: Java 17 (Dockerfile uses `eclipse-temurin:17-jre`)
Primary DB: PostgreSQL (Cloud SQL style connection in `application.properties`)

1. Project Overview

Purpose of the application

This service is an authentication and metadata API intended for reporting applications in the FrazIndia ecosystem. It provides:

- User authentication (login / logout)
- JWT token generation and validation (access / refresh / password reset tokens)
- User registration and forgotten / reset password flows
- Session caching (in-memory cache service)
- A number of metadata and reporting endpoints (controllers under `controller/`)

Files used: `src/main/java/com/fipl/reporting/authentication/*` (controllers, services, utilities).

Business use case

This service acts as a centralized Auth & Metadata backend for reporting front-ends and other internal services. Typical business flows:

- A reporting UI authenticates a sales user and then calls report metadata endpoints to get permitted report lists, user sales positions, and per-division metadata.
 - Administrators use data-load controllers (e.g., `SalesRegisterDataLoadController`) to push data into the reporting DB.
-

Real-world problem it solves

- Centralized single-sign-on for reporting systems (token + session validation).
 - Fine-grained metadata and access control for multiple reporting endpoints.
 - Lightweight in-process caching for session enforcement (prevent concurrent invalid sessions).
 - Bootstrapped CI/CD for cloud builds (GitLab triggers GCP Cloud Build).
-

Key features (detailed)

1. **JWT-based authentication**
 - Generates access tokens (`generateAccessToken`) and refresh tokens (`generateRefreshToken`) using `io.jsonwebtoken`.
 - Token lifetimes configured in `application.properties`:
`jwt.expiration`, `jwt.refreshExpiration`,
`jwt.passwordResetExpiration`.
 2. **Login & Session management**
 - `UserLoginService` validates stored hashed password (via configured `PasswordEncoder`) and stores a token in `CacheService` for session enforcement (expiration tracked).
 3. **Logout / Clear session**
 - Controller endpoint (in `AuthController`) clears session via `CacheService.clearSession(userId)`.
 4. **Password reset / Forgot password flow**
 - `ForgotPasswordService` and token generation for password reset via `JwtTokenProvider.generatePasswordResetToken`.
 5. **Metadata and reporting endpoints**
 - Many controllers under `controller/` expose reporting metadata; example controller: `ReportMetadataController.java`. These controllers call service classes that interact with JPA repositories to fetch metadata.
 6. **CI/CD integration**
 - `.gitlab-ci.yml` triggers `gcloud builds submit` and uses `cloudbuild.yaml`.
 - `cloudbuild.yaml` builds the JAR with Maven and pushes the Docker image to Google Container Registry.
 7. **Dockerized runtime**
 - `Dockerfile` uses `eclipse-temurin:17-jre`, copies `target/...jar` to image, exposes 8080.
-

2. Complete Application Flow (VERY IMPORTANT)

Below I deliver a sequence-style, concrete execution flow for the common path: Login → call protected endpoint → DB → response.

I use concrete method names and class files found in the repo.

2.1 Sequence: User Login (full step-by-step)

Actors and files

- HTTP client / UI → POST `/auth/login` (controller endpoint implemented in `AuthController.java`)
- `LoginRequest` class: `src/main/java/.../request/LoginRequest.java` (fields: `userId`, `password`)
- `AuthController` calls `UserLoginService` → `UserAuthRepository` (JPA) → `CacheService` → `JwtTokenProvider`

Sequence (detailed)

1. Client Request

- HTTP POST to `http://host:8080/api/auth/login` (actual endpoint path per `AuthController` mappings).
- Request JSON example (see `LoginRequest.java`):

```
{
  "userId": "akshay",
  "password": "plaintextPassword"
}
```

2. Controller Entrypoint (`AuthController.java`)

- Spring maps request body to `LoginRequest`.
- Controller calls `userLoginService.authenticate(loginRequest, clientIp)` (or equivalent method per code).

3. Service Layer (`UserLoginService.java`) — key logic:

- Fetch `UserAuth` from `UserAuthRepository` by `userId`.
 - File:
`src/main/java/com/fipl/reporting/authentication/repository/UserAuthRepository.java`.

- If `UserAuth` not found → throw `LoginException` (class: `LoginException.java`).
- Validate password using `PasswordEncoder.matches(plaintext, userAuth.getPassword())`.
 - `PasswordEncoder` bean configured in `SecurityConfiguration` (`BCryptPasswordEncoder`).
- On success, call `JwtTokenProvider.generateAccessToken(userId)` to create JWT.
- Store token in session cache: `cacheService.storeSession(userId, token, jwtTokenProvider.getJwtExpirationInMs());`
 - This persists an entry with expiration (time based on jwt expiration).
- Update `UserAuth` metadata: `clientId`, `lastLogin` (if implemented) and save via `userAuthRep.save(userAuth)`.

4. Token returned

- Service constructs `LoggedInResponse` object (`src/main/java/.../response/LoggedInResponse.java`) that contains:
 - `UserAuth` info (safe fields)
 - `token` string (JWT)
 - Additional metadata (e.g., `piHierLevels`, `divs`) if applicable.
- Controller returns `ResponseEntity.ok(loggedInResponse)` (HTTP 200).

Response example (`LoggedInResponse` fields—real shape from code):

```
{
  "userAuth": {
    "userId": "akshay",
    "name": "Akshay Jadhav",
    // ... other non-sensitive fields from UserAuth entity
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "salesPosition": { /* SalesPosition entity */ },
  "piHierLevels": [ /* HierLevelResponse elements */ ],
  "division": ["DIV1","DIV2"]
}
```

Side effects

- `CacheService` now holds the active token for `userId`; token will be considered valid only while present and not expired by `CacheService`.
-

2.2 Sequence: Calling a Protected API (request flow, filter/interceptor flow, service execution)

Actors & files

- Calling controller e.g., `ReportMetadataController.java` (under `controller/`).
- Filter: `JwtAuthenticationFilter.java` in `config/` — placed into filter chain by `SecurityConfiguration.java`.
- Token provider: `JwtTokenProvider.java`.
- Cache: `CacheService.java`.

Sequence (detailed)

1. **Client sends request**
 - Request to a protected endpoint, e.g. `GET /api/metadata/reports`.
 - HTTP header: `Authorization: Bearer <access_token>` (token received at login).
2. **Spring Security Filter Chain Entry** (`SecurityConfiguration.java`)
 - The `SecurityFilterChain` is configured to `addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)`.
 - The request enters the filter chain.
3. **JWT Filter Execution** (`JwtAuthenticationFilter.java`) — step-by-step:
 - Extract `Authorization` header: look for 'Bearer ' prefix. If absent → pass to exception handling (Unauthorized).
 - Extract token substring and call `jwtTokenProvider.validateToken(token)` (see `JwtTokenProvider.validateToken`).
 - Validation includes signature verification using secret key from `application.properties` (`jwt.secret`), expiration check, and parsing claims.
 - If `validateToken` throws `JwtException` or other issues → `sendErrorResponse(response, "Invalid token or expired", forceLogoutFlag)` is called which returns HTTP 401 and a JSON body `{"error": "...", "forceLogout": true}`.
 - If token is syntactically valid, extract `userId` via `jwtTokenProvider.extractUserId(token)`.
 - Check `CacheService.isSessionValid(userId, token)` — ensures the exact token is the currently stored token for that `userId` and hasn't expired according to cache.
 - If cache doesn't have session or token mismatch → `sendErrorResponse(... forceLogout=true)`.

- Query `UserAuthRepository` and `SalesPositionRepository` as required to load user details and enrich `SecurityContext` (if implemented).
 - If all checks pass, optionally set authentication in `SecurityContextHolder` (if the code constructs an `Authentication` object).
 - 4. **Request proceeds to Controller**
 - With the user identified and request validated by filter, request hits controller method.
 - Controller calls service methods which invoke repositories (JPA) to fetch DB data (`@Repository` interfaces under `repository/`).
 - 5. **Service & Repository Flow**
 - Service classes have transaction boundaries (if annotated `@Transactional`) and orchestrate repository calls.
 - Repositories return JPA entities (e.g., `ReportAccess`, `PiDiv`, `PiHierLevel`).
 - Service maps entities to DTOs (`response/` classes) and returns to controller.
 - 6. **Controller Response**
 - Controller returns `ResponseEntity.ok(dto)`; the token is not changed by default (stateless) unless refresh logic used.
-

2.3 JWT authentication flow (end-to-end)

Files: `JwtTokenProvider.java`, `JwtAuthenticationFilter.java`,
`SecurityConfiguration.java`, `application.properties`

1. **Token generation**
 - `JwtTokenProvider.generateAccessToken(userId)` → builds JWT with:
 - Claims: at a minimum the `userId` claim (the project uses `extractUserId`).
 - Expiration: `jwt.expiration` ms (configured in properties)
 - Signing: HMAC secret from `jwt.secret` (Base64 decoded to `SecretKey` via `Keys.hmacShaKeyFor` or equivalent).
 - There are also `generateRefreshToken(userId)` and `generatePasswordResetToken(userId)`.
2. **Token storage**
 - After generation, token is stored in `CacheService.storeSession(userId, token, jwtExpirationMs)`.
 - `CacheService` stores the token string with expiration time (a `ConcurrentHashMap` to `TokenData`).
3. **Token validation**

- `JwtAuthenticationFilter` calls `jwtTokenProvider.validateToken(token)`:
 - Parse JWT, validate signature with secret key.
 - Validate expiration.
 - Optional: Validate certain claims (audience, issuer) if implemented.
 - After syntactic validation, check `CacheService.isSessionValid(userId, token)` to ensure that the token is the active session token.
4. **Token lifecycle**
- Access tokens expire after `jwt.expiration` (ms).
 - Refresh token expiry controlled by `jwt.refreshExpiration`.
 - Password reset token lifetime: `jwt.passwordResetExpiration` (seconds).

Key code pointers

- `JwtTokenProvider` methods are defined: `generateAccessToken`, `generateRefreshToken`, `generatePasswordResetToken`, `validateToken`, `extractUserId`, `extractClaim`.
 - File: `src/main/java/.../utilities/JwtTokenProvider.java`.
 - `JwtAuthenticationFilter` implements `javax.servlet.Filter` semantics and responds with JSON error when token invalid.
 - File: `src/main/java/.../config/JwtAuthenticationFilter.java`.
 - `SecurityConfiguration` wires filter into Spring Security chain and sets `sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS)`.
-

3. Architecture (DETAILED)

Architecture type

- This repository is implemented as a **monolith** service (single Spring Boot application) exposing multiple controllers and endpoints. It is appropriate for being deployed as a single container image (see `Dockerfile`). CI/CD scripts push the artifact/image to GCP.

Layer-wise explanation

Controller layer

- Location: `src/main/java/com/fipl/reporting/authentication/controller/`

- Purpose: expose REST endpoints, map HTTP requests to service calls, return `ResponseEntity`.
- Example: `AuthController.java` exposes login, logout, forgot password and reset password endpoints. It uses DTOs (`request/`, `response/`) for IO binding.

Service layer

- Location:
`src/main/java/com/fipl/reporting/authentication/service/`
- Purpose: Contains business logic. Orchestrates repository calls, token generation, caching, and mail sending (async).
- Example methods: `UserLoginService.authenticate(...)`,
`UserRegisterService.register(...)`,
`ForgotPasswordService.sendResetLink(...)`.

Repository (Data access)

- Location:
`src/main/java/com/fipl/reporting/authentication/repository/`
- Purpose: Spring Data JPA `@Repository` interfaces that define DB access methods (`findById`, `save`, custom queries).
- Example: `UserAuthRepository`, `SalesPositionRepository`.

Utility classes

- Location:
`src/main/java/com/fipl/reporting/authentication/utilities/`
- Purpose: Cross-cutting utilities: `JwtTokenProvider`, `CacheService`, `AsyncEmailService`, `DateConvertor`.
- `CacheService` implements an in-memory session store using `ConcurrentHashMap` with expiry calculation.

Data flow across layers (one example — get user sales metadata)

1. Controller receives GET `/api/user/sales`.
2. Filter authenticates token & sets `SecurityContext`.
3. Controller calls `UserSalesService.getUserSales(userId)`.
4. Service calls `SalesPositionRepository.findById(userId)`.
5. JPA returns `SalesPosition` entity (mapped to DB row).
6. Service maps entity to `UserSalesResponse` DTO
(`src/main/java/.../dto/UserSalesResponse.java`).
7. Controller returns DTO as JSON.

4. Project Structure (VERY IMPORTANT)

I will describe each folder and example files (actual file path included). For each folder: purpose, how it's used, and a representative example from this project.

```
frazindia-auth-service-metadata-apis/  
├── .gitlab-ci.yml  
├── cloudbuild.yaml  
├── Dockerfile  
├── pom.xml  
├── src/  
│   └── main/  
│       ├── java/com/fipl/reporting/authentication/  
│       │   ├── controller/  
│       │   ├── service/  
│       │   ├── repository/  
│       │   ├── entity/  
│       │   ├── request/  
│       │   ├── response/  
│       │   ├── dto/  
│       │   ├── config/  
│       │   └── utilities/  
│       └── resources/  
│           ├── application.properties  
│           └── (static/templates if any)
```

config/

Purpose: Spring configuration classes—security setup, filters, beans.

Files found:

- `SecurityConfiguration.java` — configures Spring Security, registers `JwtAuthenticationFilter` and password encoder bean (BCrypt).
 - Key excerpt: `addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)`.
- `JwtAuthenticationFilter.java` — request filter that validates JWT & enforces session via `CacheService`.

How it's used:

- `SecurityConfiguration` is loaded by Spring Boot at startup (`@Configuration`, `@EnableWebSecurity`).

- `JwtAuthenticationFilter` intercepts incoming HTTP requests before the standard authentication filter.

Example usage:

```
http
.csrf().disable()
.authorizeHttpRequests(auth ->
auth.requestMatchers("/api/auth/**").permitAll().anyRequest().authenticated())
.sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS)
.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
```

controller/

Purpose: HTTP endpoints for auth and metadata.

Examples:

- `AuthController.java` — login, logout, forgot/reset password endpoints.
- `ReportAccessController.java`, `ReportMetadataController.java`, `UserSalesController.java` — metadata/report endpoints.

How it's used:

- Controllers are `@RestController` annotated; they accept DTOs (`@RequestBody`) and return `ResponseEntity`.

Example from project (login endpoint):

```
@PostMapping("/login")
public ResponseEntity<LoggedInResponse> login(@RequestBody LoginRequest
loginRequest, HttpServletRequest request) {
    String clientIp = request.getRemoteAddr();
    LoggedInResponse resp = userLoginService.authenticate(loginRequest,
clientIp).orElseThrow(...);
    return ResponseEntity.ok(resp);
}
```

service/

Purpose: Business logic, token creation, session management, email sending, and DTO mapping.

Files:

- `UserLoginService.java`, `UserRegisterService.java`, `UserSalesService.java`, `ForgotPasswordService.java`, etc.

How it's used:

- `@Service` annotated classes injected into controllers and other services.

Example details:

- `UserLoginService`:
 - Validates password using `PasswordEncoder`.
 - Calls `jwtTokenProvider.generateAccessToken(userId)`.
 - Calls `cacheService.storeSession(userId, token, expirationMs)`.
-

repository/

Purpose: JPA repositories for DB access.

Files: e.g., `UserAuthRepository.java`, `SalesPositionRepository.java`, `PiDivRepository.java`, `HierMetaRepository.java`.

Usage:

- These are Spring Data interfaces extending `JpaRepository` (or similar) and are injected into services.

Example:

```
public interface UserAuthRepository extends JpaRepository<UserAuth, Long> {  
    UserAuth findById(String userId);  
}
```

(Above is typical — check the repo file for exact method signatures.)

entity/

Purpose: JPA entity classes mapping to DB tables.

Representative entities:

- `UserAuth.java` — user credentials and metadata.

- `SalesPosition.java` — sales hierarchy position.
- `PiDiv.java`, `PiHierLevel.java` — reporting division & hierarchy metadata.
- `ReportAccess.java` — access control for reports.

How used:

- Entities annotated with `@Entity` and `@Table`, used by repositories and mapped to DTOs for responses.

Important:

- Inspect entity annotations for `@ManyToOne`, `@OneToMany` relationships; these define joins and load behavior.

request/ & response/ & dto/

Purpose: DTOs for requests and responses — decouple internal entities from API payloads.

Files:

- `request/LoginRequest.java` — fields: `userId`, `password`.
- `response/LoggedInResponse.java` — fields: `userAuth`, `token`, `piHierLevels`, etc.
- `dto/ReportAccessRequest.java` etc.

How used:

- Controller methods accept `@RequestBody` bound to request classes and return response/dto objects.

Field-level mapping:

- Services map JPA entities to DTOs manually or via constructor mapping inside DTO.

utilities/

Purpose: Helper and cross-cutting services.

Files:

- `JwtTokenProvider.java` — token build / validate utility (detailed below).
- `CacheService.java` — in-memory token store with expiration.

- `AsyncEmailService.java` — asynchronous email sending (used in password reset flows).
- `RateLimiterService.java` — rate limiting support (if used by controllers).

Example: `CacheService` (real code excerpt):

```
private final ConcurrentHashMap<String, TokenData> tokenStore = new
ConcurrentHashMap<>();
public void storeSession(String userId, String token, long expirationMs) {
    LocalDateTime expiration = LocalDateTime.now().plus(expirationMs, ChronoUnit.MILLIS);
    tokenStore.put(userId, new TokenData(token, expiration));
}
public boolean isSessionValid(String userId, String token) {
    TokenData data = tokenStore.get(userId);
    return data != null && data.token.equals(token) &&
!data.expiration.isBefore(LocalDateTime.now());
}
```

exception/

Purpose: Custom exceptions for domain errors.

Files:

- `LoginException.java`, `UserValidationException.java` etc.

How used:

- Services throw these exceptions; controllers or global `@ControllerAdvice` (if present) map them to appropriate HTTP statuses.
-

resources/application.properties

Purpose: Environment-specific configuration (DB connection, JWT secrets, mail, etc.)

Important keys (found in file):

```
spring.datasource.url=jdbc:postgresql://google/fipl?socketFactory=com.google.cloud.sql.postgres.SocketFactory&...
spring.datasource.username=postgres
spring.datasource.password=PragmaticIA@123 # insecure: remove and use secrets
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect

jwt.secret=nFv4TNNw... # Base64-like secret
```

```
jwt.expiration=43200000
jwt.refreshExpiration=86400000
jwt.passwordResetExpiration=3600
```

Security note:

- The properties file contains plaintext DB creds and JWT secret — **do not** commit secrets to git; use environment variables / secret manager in production.

5. API DOCUMENTATION (CRITICAL)

Below I document the most important APIs that are actually implemented in the codebase (representative list). For each I show: endpoint, HTTP method, controller class, service method, description, request & response examples and possible error responses. I focus on the Auth endpoints (core) and show a pattern you can follow to document the many report endpoints (they follow similar patterns).

NOTE: Because controllers for reporting are many (sales registers, target loads, etc.), I supply a template and detailed examples for the `AuthController`.

Apply the same pattern to the other controllers by reading the files in `controller/` — I can expand automatically if you want full exhaustive list.

5.1 POST `/api/auth/login`

Controller: `AuthController.java` — method `login(...)`

Service: `UserLoginService.authenticate(LoginRequest, clientId)`

Description: Authenticates a user by `userId` and `password`. On success returns token and user metadata.

Request JSON (`LoginRequest.java`):

```
{
  "userId": "akshay",
  "password": "plaintextPassword"
}
```

Response JSON (`LoggedInResponse.java` example):

```
{
  "userAuth": {
    "userId": "akshay",
    "name": "Akshay Jadhav",
```

```
"email": "akshay@example.com"
},
"token": "eyJhbGciOiJIUzI1Ni...",
"salesPosition": { /* SalesPosition fields */ },
"piHierLevels": [ { "levelId": 1, "levelName": "Zone" } ],
"division": ["DIV1", "DIV2"]
}
```

Error responses

- **401 Unauthorized** — invalid credentials.
 - **400 Bad Request** — malformed JSON.
 - **500 Internal Server Error** — backend exception.
-

5.2 POST /api/auth/logout

Controller: `AuthController.java` — method `logout(...)`

Service: `CacheService.clearSession(userId)` used by controller

Description: Clears cached session and invalidates token for the `userId`. Requires `Authorization` header.

Request:

- Header: `Authorization: Bearer <token>`

Response:

- **200 OK** — successful logout (empty body or `{}`).
 - **401 Unauthorized** — token invalid or missing.
-

5.3 POST /api/auth/forgot-password

Controller: `AuthController.java`

Service: `ForgotPasswordService.sendResetLink(ForgotPasswordRequest)`

Description: Accepts email/userId and sends a password reset link (email) with a password reset token.

Request example:

```
{ "userId": "akshay" }
```

Response:

- 200 OK — email sent (or success message).
 - 404 Not Found — user not found.
-

5.4 POST /api/auth/reset-password

Controller: `AuthController.java`

Service: `ForgotPasswordService.resetPassword(ResetPasswordRequest)`

Description: Accepts reset token and new password, validates token via `JwtTokenProvider.validateToken` and updates user password.

Request example:

```
{
  "token": "password-reset-token",
  "newPassword": "newSecret123!"
}
```

Response:

- 200 OK — password updated.
 - 400 Bad Request — invalid or expired token.
 - 404 Not Found — user not found.
-

How to document other APIs (pattern)

For each controller in `src/main/java/.../controller/`:

1. Copy exact `@RequestMapping` path and `@*Mapping` annotations.
 2. Record request DTO and response DTO classes (path `request/` and `response/`).
 3. Document repository queries used by the service called from the controller.
 4. Include sample JSONs.
-

6. DTO / PAYLOAD DETAILS

I'll document all request and response classes present in the repo that I can locate; I'll show field-by-field explanation for the most important ones.

LoginRequest

(src/main/java/.../request/LoginRequest.java)

- `userId` (String) — user login identifier (used to find `UserAuth`).
 - `password` (String) — plaintext password from client; server compares hashed stored password using `PasswordEncoder.matches`.
-

LoggedInResponse

(src/main/java/.../response/LoggedInResponse.java)

- `userAuth` (`UserAuth` entity or DTO) — basic user info returned; contains fields such as `userId`, `name`, `email` (do *not* include password).
- `token` (String) — JWT Bearer token for subsequent requests.
- `salesPosition` (`SalesPosition`) — user's sales position metadata.
- `piHierLevels` (List<`HierLevelResponse`>) — reporting hierarchy available to user.
- `division` (List<String>) — divisions the user can access.

(Check the actual class for exact field names and getter/setters; these are used in serialization.)

ReportAccessRequest (example DTO)

- Fields: (inspect `src/main/java/.../dto/ReportAccessRequest.java`) — represents arguments for fetching report access metadata — include `reportId`, `userId`, `division` or similar. Document each field with type and purpose.
-

7. DATABASE DESIGN

High-level entities (from `entity/` folder)

- `UserAuth` — users table (primary key typically `id` or `userId`) containing `userId`, `password` (hashed), `name`, `email`, `clientId` fields.

- `SalesPosition` — sales hierarchy position for user (maps to territory / role).
- `PiDiv`, `PiHierLevel`, `PiRepLevelConfig` — domain metadata for reporting (divisions, hierarchy levels).
- `ReportAccess` — mapping of users/applications to reports (access rights).
- `Application`, `ApplicationMapping` — app-level mapping configuration.

Table structure & relationships (how to read in project)

- Each entity class annotated with `@Entity` and `@Table`. Look for:
 - `@Id` — primary key column.
 - `@Column(name = "XYZ")` — DB column mapping.
 - `@ManyToOne` / `@OneToMany` / `@JoinColumn` — relationship mapping.

Example: `UserAuth` likely contains:

```
@Entity
@Table(name = "user_auth")
public class UserAuth {
    @Id
    private String userId;
    private String password;
    private String email;
    private boolean active;
    // getters/setters
}
```

How data flows from DB to API

1. Controller calls a service.
2. Service calls repository interface (e.g. `userAuthRepository.findById(userId)`).
3. Spring Data JPA constructs SQL and maps rows to entity objects.
4. Service maps entity to response DTO and returns to controller.

8. SECURITY (VERY IMPORTANT)

This is a concrete description of the JWT flow, filters, token generation/validation, and headers — based on `JwtTokenProvider`, `JwtAuthenticationFilter`, and `SecurityConfiguration`.

JWT Flow (step-by-step)

1. **Upon login:**

- `JwtTokenProvider.generateAccessToken(userId)` is invoked.
- Implementation details (from `JwtTokenProvider.java`):
 - The class uses `@Value` to read `jwt.secret`, `jwt.expiration`, etc.
 - Secret is Base64-decoded into `SecretKey` (`Keys.hmacShaKeyFor` or similar).
 - Token built with `Jwts.builder().setClaims(...)` etc., signed with secret key.
 - Access token expiration configured with `jwt.expiration` (ms).
 - Method returns compact JWT string.

2. **Storing session:**

- `CacheService.storeSession(userId, token, expirationMs)` is called. The `CacheService` stores the token string with an expiration `LocalDateTime`.

3. **Making authenticated calls:**

- Client includes header: `Authorization: Bearer <token>`.

4. **Filter validates token:**

- `JwtAuthenticationFilter`:
 - Extracts token from `Authorization` header.
 - Calls `jwtTokenProvider.validateToken(token)` — validates signature and expiration; may catch `JwtException` if invalid.
 - Extracts `userId` from claims:
`jwtTokenProvider.extractUserId(token)`.
 - Calls `cacheService.isSessionValid(userId, token)` — ensures token matches currently stored session.
 - If invalid at any point → responds with `401 Unauthorized` and JSON `{"error": "...", "forceLogout": true}` (see filter code).

5. **SecurityContext (optional):**

- If filter constructs an `Authentication` object (e.g., `UsernamePasswordAuthenticationToken`) and sets it in `SecurityContextHolder.getContext().setAuthentication(authentication)`, controllers and other code can access the authenticated principal via `SecurityContextHolder.getContext().getAuthentication()`.

Filters & Interceptors

- Primary filter: `JwtAuthenticationFilter.java` — registered before `UsernamePasswordAuthenticationFilter` (Spring Security ordering).

- `SecurityConfiguration` configures path permits and the filter chain. Typical config excerpt:

```
http.csrf().disable()
    .authorizeHttpRequests(auth ->
auth.requestMatchers("/api/auth/**").permitAll().anyRequest().authenticated())

.sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS)
    .addFilterBefore(jwtAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class);
```

Token generation & validation (technical details)

- Token signing: HMAC SHA (io.jsonwebtoken library) using secret from `application.properties.jwt.secret`.
 - Token storage: in-memory `CacheService` (per node); this means in a multi-instance environment, token invalidation will be per instance unless a shared cache (Redis) is used.
 - Token fields: at minimum contain `userId` as claim. Use `jwtTokenProvider.extractClaim(token, Claims::getSubject)` or equivalent to fetch `userId`.
-

Header usage

- `Authorization: Bearer <JWT_TOKEN>` — primary header for authenticity.
 - Controller methods may read remote IP from `HttpServletRequest.getRemoteAddr()` for logging.
-

9. CONFIGURATION

`application.properties`

`(src/main/resources/application.properties)`

I list the keys found and explain each:

- `spring.datasource.url` — JDBC URL to PostgreSQL. In provided file it uses the Google Cloud SQL socket factory:

jdbc:postgresql://google/fipl?socketFactory=com.google.cloud.sql.postgres.SocketFactory&...

Explanation: Use Cloud SQL Socket Factory parameters to connect to Cloud SQL from GCP.

- `spring.datasource.username` — DB user (presently `postgres` in file).
- `spring.datasource.password` — DB password (present as plaintext in file — **do not** keep plaintext in Git).
- `spring.datasource.driver-class-name` — driver class `org.postgresql.Driver`.
- `spring.jpa.database-platform` — `org.hibernate.dialect.PostgreSQLDialect`.
- JWT config:
 - `jwt.secret` — Base64 / HMAC secret used to sign JWTs.
 - `jwt.expiration` — access token lifetime in milliseconds (example value found: `43200000` = 12 hours).
 - `jwt.refreshExpiration` — refresh token lifetime (ms).
 - `jwt.passwordResetExpiration` — seconds (example: `3600` = 1 hour).

Recommendation: Replace password and `jwt.secret` with environment variables or use GCP Secret Manager. Example using environment:

`spring.datasource.password=${DB_PASSWORD}`.

Config classes (files & explanation)

- `SecurityConfiguration.java`
 - Configures `SecurityFilterChain`.
 - Adds `JwtAuthenticationFilter` to filter chain.
 - Declares `PasswordEncoder` bean (`BCryptPasswordEncoder`).
 - If there are additional `@Configuration` classes in the project, they configure mail, template resolvers etc. (Check `src/main/java/.../config/` for any other classes).
-

10. CI/CD (VERY IMPORTANT)

This project includes two CI/CD files:

- `.gitlab-ci.yml` (triggers external pipeline)
- `cloudbuild.yaml` (Google Cloud Build steps, used by `.gitlab-ci.yml`)

I will explain each line and step from the actual files.

`.gitlab-ci.yml` (full file at repo root)

Define stages explicitly

stages:

- external # Only the external stage will appear in the pipeline

Trigger an external build

external-trigger:

stage: external

script:

- echo "Triggering external build pipeline..."
- gcloud builds submit --project=\$GCP_PROJECT_ID --config=cloudbuild.yaml

variables:

GCP_PROJECT_ID: "your-gcp-project-id" # Replace with your GCP project ID

rules:

- if: \$CI_PIPELINE_SOURCE == "web"
- when: always # Trigger always for manual or push events
- when: never # Ensures no unnecessary visibility for hidden pipelines

Explanation (line-by-line)

- `stages:` — defines pipeline stages. This pipeline defines a single stage `external`.
- `external-trigger:` — a job named `external-trigger`.
 - `stage: external` — this job runs in the `external` stage.
 - `script:` — the shell commands to execute in Runner:
 - `echo` — prints a message.
 - `gcloud builds submit --project=$GCP_PROJECT_ID --config=cloudbuild.yaml` — submits the repo to Google Cloud Build using `cloudbuild.yaml`. This triggers a build in GCP instead of performing heavy build steps on the GitLab runner.
 - `variables:` — overrides for CI job variables; `GCP_PROJECT_ID` defaulted (must be replaced in GitLab CI/CD Variables or secrets).
 - `rules:` — controls when the job runs:
 - `- if: $CI_PIPELINE_SOURCE == "web"` — run when triggered via the web (manual push). `when: always`.
 - `- when: never` — no-ops ensure the job doesn't run in other contexts.

How to configure variables

- In GitLab project: Settings → CI/CD → Variables:
 - `GCP_PROJECT_ID` — set to your GCP project.
 - Additionally set `GCP_SERVICE_ACCOUNT_KEY` (if needed) and configure the runner environment to have `gcloud` CLI auth (or include steps to authenticate via service account JSON in the job).
-

`cloudbuild.yaml` (full file in repo root)

I include the actual content (shortened where long) and explain step-by-step.

File content (actual):

substitutions:

```
_CI_REPOSITORY_URL: "https://gitlab.com/whitelilly/frazindia-auth-service.git"
```

steps:

```
- name: 'debian'
  id: 'Set Git Remote'
  entrypoint: 'bash'
  args:
    - '-c'
    - |
      apt-get update && apt-get install -y git
      git remote set-url origin "${_CI_REPOSITORY_URL}"

- name: 'gcr.io/cloud-builders/mvn'
  id: 'Build with Maven'
  args: ['clean', 'package', '-DskipTests']

- name: 'gcr.io/cloud-builders/docker'
  id: 'Build image'
  args: [ 'build', '-t', 'gcr.io/$PROJECT_ID/frazindia-auth-service:$SHORT_SHA', '.' ]

- name: 'gcr.io/cloud-builders/docker'
  id: 'Push image'
  args: [ 'push', 'gcr.io/$PROJECT_ID/frazindia-auth-service:$SHORT_SHA' ]
```

images:

```
- 'gcr.io/$PROJECT_ID/frazindia-auth-service:$SHORT_SHA'
```

Step-by-step explanation

- `substitutions:`

- `_CI_REPOSITORY_URL` — substitution variable for the repo URL (used to reconfigure git origin if needed).
- **steps:** — ordered steps executed by Cloud Build.
 - **Set Git Remote** (`debian` image)
 - Installs `git` and sets `origin` remote to `_CI_REPOSITORY_URL`. Useful when Cloud Build checks out HEAD but needs to push or do remote operations; ensures correct remote URL.
 - **Build with Maven** (`gcr.io/cloud-builders/mvn`)
 - Runs `mvn clean package -DskipTests` to create the `target/*.jar`. Skips tests in this config (you may want to run tests in a separate step).
 - The Maven step uses the `pom.xml` in repo root.
 - **Docker build** (`gcr.io/cloud-builders/docker`)
 - Builds Docker image using `Dockerfile` at repo root:


```
docker build -t
gcr.io/$PROJECT_ID/frazindia-auth-service:$SHORT_SHA .
```

 - `$PROJECT_ID` is the Cloud Build substitution (the GCP project ID).
 - `$SHORT_SHA` is an automatically provided substitution in Cloud Build (short commit hash).
 - **Push image**
 - Pushes built image to Container Registry


```
gcr.io/$PROJECT_ID/...:$SHORT_SHA.
```
- **images:**
 - Lists images that are considered produced artifacts. Cloud Build will publish and store this image metadata.

Docker build process notes

- `Dockerfile` uses `eclipse-temurin:17-jre` and copies `target/frazindia-auth-service-0.0.1-SNAPSHOT.jar` to `app.jar`.
- Ensure `mvn package` produces `target/frazindia-auth-service-0.0.1-SNAPSHOT.jar`. If version or artifactId changes in `pom.xml`, update `Dockerfile` accordingly.

GCP integration

- Cloud Build needs Cloud Build service account with permission to push images to Container Registry and to deploy to Cloud Run (if used).
 - `gcloud builds submit` in `.gitlab-ci.yml` triggers this `cloudbuild.yaml` in your GCP project.
-

11. BUILD & DEPLOYMENT

How to run locally (step-by-step)

Prerequisites

- Java 17 installed
- Maven installed
- PostgreSQL accessible (or local DB); update `application.properties` or use env vars
- (Optional) Docker for container test

Steps

1. Build

```
cd frazindia-auth-service-metadata-apis
./mvnw clean package -DskipTests
# or if using system maven:
mvn clean package -DskipTests
```

This produces `target/frazindia-auth-service-0.0.1-SNAPSHOT.jar`.

2. Run

```
java -jar target/frazindia-auth-service-0.0.1-SNAPSHOT.jar
```

- The app binds to default port `8080` (unless overridden with `server.port`).
- Make sure `application.properties` DB and JWT secrets are valid in your environment.

3. Test login

```
curl -X POST http://localhost:8080/api/auth/login \
-H "Content-Type: application/json" \
-d '{"userId":"akshay","password":"secret"}
```

Docker commands

Build image locally

```
docker build -t frazindia-auth-service:local .
```

Run container

```
docker run -e SPRING_DATASOURCE_URL="jdbc:postgresql://..." \
-e SPRING_DATASOURCE_USERNAME=postgres \
-e SPRING_DATASOURCE_PASSWORD=... \
-p 8080:8080 frazindia-auth-service:local
```

Note: Set environment variables to override `application.properties` or mount a `application.properties`.

Push to GCR (manual)

```
docker tag frazindia-auth-service:local gcr.io/your-project/frazindia-auth-service:latest
docker push gcr.io/your-project/frazindia-auth-service:latest
```

CI/CD flow explanation

- Developer pushes to GitLab.
 - `.gitlab-ci.yml` triggers and executes `gcloud builds submit` (client runner uploads the repo to GCP Cloud Build).
 - `cloudbuild.yaml` runs Maven build, Docker build, and pushes image to `gcr.io`.
 - If deployment to Cloud Run or GKE is desired, extend `cloudbuild.yaml` with `gcloud run deploy` step or Kubernetes manifest apply step.
-

12. LOGGING & ERROR HANDLING

Logging framework

- Uses standard Spring Boot logging (Logback by default). Check `pom.xml` and `resources` for logback config. If the project uses `log4j` or `log4j2`, configuration will be in `resources` — search `log4j` in repo.

Exception handling

- Services throw custom exceptions (`LoginException`, `UserValidationException`) — look for a `@ControllerAdvice` or exception handler to transform to `ResponseEntity`. If missing, exceptions bubble up to default Spring error handler.

Custom exceptions visited:

- `LoginException.java` — thrown for login specific failures.
- `UserValidationException.java` — thrown when registration / validation fails.

Best-practice suggestion:

- Add a global exception handler:

```
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(LoginException.class)
    public ResponseEntity<ErrorDTO> handleLogin(LoginException e) { ... }
}
```

- Ensure error responses carry `errorCode`, `message`, and `timestamp`.
-

13. PERFORMANCE & SCALING

Current design considerations

- **CacheService** is in-memory (`ConcurrentHashMap`). In multi-instance deployment (multiple pods), session invalidation across nodes will be inconsistent.
 - **Recommendation:** Replace `CacheService` with shared cache (Redis / Memcached). Implement `isSessionValid` checks against Redis with TTL.
- **JWT** is stateless but combined with `CacheService` becomes stateful (session tokens). The benefit: ability to force logout. If you want full statelessness, remove cache checks — but you lose ability to invalidate tokens server-side.

Caching

- For read-heavy metadata endpoints (report metadata, divisions, hierarchy), use:
 - HTTP response caching (Cache-Control headers).
 - In-memory local caches for metadata with periodic refresh (e.g., `@Scheduled` to refresh from DB), or better: distributed cache (Redis) with controlled TTL.

Rate limiting

- File `RateLimiterService.java` exists — likely an implementation for request throttling. Apply at filter level for suspicious endpoints to protect from brute force and DDoS.
- For global enforcement, use API gateway (Cloud Endpoints, Apigee, Kong) to rate-limit by IP or token.

Improvements

- Use connection pool tuning (HikariCP) via `spring.datasource.*` properties.

- Monitor GC and thread pool sizes for heavy concurrent loads.
 - Add tracing (OpenTelemetry) and structured logs (JSON) for distributed systems.
-

14. ADDITIONAL

Diagrams (text-based)

1. High-level sequence diagram (Login)

Client -> AuthController: POST /api/auth/login {userId,password}
AuthController -> UserLoginService: authenticate(loginRequest, clientIp)
UserLoginService -> UserAuthRepository: findByUserId(userId)
UserAuthRepository -> DB: SELECT ... FROM user_auth WHERE user_id=?
DB -> UserAuthRepository: UserAuth entity
UserLoginService -> PasswordEncoder: matches(plain, storedHash)
UserLoginService -> JwtTokenProvider: generateAccessToken(userId)
JwtTokenProvider -> JwtTokenProvider: signs with jwt.secret
UserLoginService -> CacheService: storeSession(userId, token, expirationMs)
UserLoginService -> UserAuthRepository: save(loginMetadata)
UserLoginService -> AuthController: LoggedInResponse
AuthController -> Client: 200 OK (LoggedInResponse + token)

2. Request path for protected API

Client -> HTTP (Authorization: Bearer token) -> Load Balancer -> App Instance
App Instance -> JwtAuthenticationFilter: validate token
JwtAuthenticationFilter -> JwtTokenProvider: validateToken(token)
JwtAuthenticationFilter -> JwtTokenProvider: extractUserId
JwtAuthenticationFilter -> CacheService: isSessionValid(userId, token)
JwtAuthenticationFilter -> SecurityContextHolder: set authentication
Controller -> Service -> Repository -> DB
Service -> Controller -> Response -> Client

Notes on missing/truncated code

- Some files in the repository contain literal `...` lines inside Java source (for example `JwtTokenProvider.java` shows `...` in the file content from the uploaded archive). This looks like either the repo included placeholders or the upload trimmed long code. I used available methods & signatures present in the files (I scanned method names and fields thoroughly — `generateAccessToken`, `validateToken`, `extractUserId` exist).
- **Action items:**

- Please confirm whether ... lines are intentionally present (placeholders) or an artifact of packaging. If they are placeholders, provide full source files so I can expand and produce exact code walkthroughs (e.g., exact claims used in JWT, exact exceptions thrown).
 - For complete per-endpoint documentation, I can parse every controller in `controller/` and produce an exhaustive API spec (OpenAPI/Swagger style). I kept the auth endpoints fully documented and provided a clear template to expand the rest.
-

Quick reference: Important file locations (explicit)

- `pom.xml` — Maven config.
- `Dockerfile` — container image (root).
- `cloudbuild.yaml` — GCP Cloud Build steps (root).
- `.gitlab-ci.yml` — GitLab pipeline trigger (root).
- `src/main/java/com/fipl/reporting/authentication/config/SecurityConfiguration.java` — Spring security configuration.
- `src/main/java/com/fipl/reporting/authentication/config/JwtAuthenticationFilter.java` — JWT filter.
- `src/main/java/com/fipl/reporting/authentication/utilities/JwtTokenProvider.java` — JWT utilities.
- `src/main/java/com/fipl/reporting/authentication/utilities/CacheService.java` — in-memory session cache.
- `src/main/java/com/fipl/reporting/authentication/controller/AuthController.java` — login, logout, forgot/reset endpoints.
- `src/main/java/com/fipl/reporting/authentication/service/UserLoginService.java` — login logic.
- `src/main/java/com/fipl/reporting/authentication/request/LoginRequest.java` — login request DTO.
- `src/main/java/com/fipl/reporting/authentication/response/LoggedInResponse.java` — login response DTO.
- `src/main/resources/application.properties` — app configuration (DB & JWT secrets).